

地面站 WebSocket 模拟客户端

本目录提供一个完全独立的 WebSocket 客户端，用于模拟当前地面站向上位机回复协议消息，遵循 `无人机与上位机通信协议V2.0.docx`：

- 连接成功后订阅 `drone/{clientNo}/command`；
- 向 `drone/{clientNo}/status` 发布命令确认和状态；
- 完成 `SYSTEM → SUBSCRIBE → SUB_ACK` 后先发送待机 01；
- 在线期间每 1 秒发送电量 0A 和位置 0B；
- 收到巡检航线 02 后保存航点并回复确认、状态 02；
- 收到执行巡检 03 后模拟起飞、逐点巡检和降落，逐点发送 09，最后发送 08；
- 支持当前地面站使用的 01 起飞、05 返航、06 降落、07 紧急停机；
- 断线后自动重连。

1. 安装 Python (Windows)

1. 打开 <https://www.python.org/downloads/windows/>，下载 Python 3 的 64 位安装程序。要求 Python 3.10 或更高版本。
2. 运行安装程序，勾选 **Add python.exe to PATH**，选择 **Install Now**。
3. 打开新的 PowerShell，确认安装：

```
1 | py --version
```

2. 安装依赖

进入本目录，创建独立环境并安装 `requirements.txt`。

Windows PowerShell：

```
1 | cd gcs-websocket-client
2 | py -m venv .venv
3 | .\venv\Scripts\Activate.ps1
4 | python -m pip install -r requirements.txt
```

如果 PowerShell 阻止激活，可不激活环境，直接执行：

```
1 | .\venv\Scripts\python.exe -m pip install -r requirements.txt
```

3. 运行

地址是 `ws://<服务器IP>:8581/ws`，以现场提供值为准

Windows PowerShell：

```

1 | .\venv\Scripts\python.exe .\gcs-websocket-client.py `
2 |     --url ws://192.168.80.10:8581/ws `
3 |     --client-no UAV01001

```

不传参数时，程序自动连接 `ws://127.0.0.1:8581/ws`，编号为 `UAV01001`。URL 和编号既可以用上述参数修改，也可以直接修改源代码顶部的 `DEFAULT_WEBSOCKET_URL` 和 `DEFAULT_CLIENT_NO`

```

dev > integration > gcs-websocket-client > gcs-websocket-client.py > ...
1 | #!/usr/bin/env python3
2 | """独立的上位机 WebSocket 协议 V2.0 地面站模拟客户端。
3 |
4 | 本文件只连接 WebSocket、接收控制主题命令并向状态主题发布模拟回复。
5 | """
6 |
7 | from __future__ import annotations
8 |
9 | import argparse
10 | import asyncio
11 | import contextlib
12 | import json
13 | import logging
14 | import math
15 | from collections.abc import Awaitable, Callable, Mapping
16 | from copy import deepcopy
17 | from typing import Any
18 | from urllib.parse import urlparse
19 |
20 | # 默认值可直接修改，也都能被同名命令行参数覆盖。
21 | DEFAULT_WEBSOCKET_URL = "ws://127.0.0.1:8581/ws"
22 | DEFAULT_CLIENT_NO = "UAV01001"
23 | DEFAULT_VIDEO_PATH = "/home/share/test.mp4"
24 | DEFAULT_JPG_DIRECTORY = "/home/share/jpg"
25 | DEFAULT_POINT_PICTURE = "/home/share/jpg/test.jpg"
26 |
27 | # 与当前地面站的航点输入边界保持一致；本模块不会把航点发给真实飞控。
28 | MAX_WAYPOINT_COUNT = 256
29 | WAYPOINT_HORIZONTAL_LIMIT_METERS = 10_000.0
30 | WAYPOINT_MINIMUM_Z_METERS = 0.1
31 | WAYPOINT_MAXIMUM_Z_METERS = 50.0
32 | LOW_POWER_PERCENTAGE = 20.0
33 |
34 | COMMAND_LABELS = {
35 |     "01": "起飞",
36 |     "02": "巡检任务下发",
37 |     "03": "执行巡检任务",
38 |     "05": "一键返航",
39 |     "06": "降落",
40 |     "07": "紧急停机",
41 | }
42 |
43 | REQUIRED_POINT_FIELDS = (
44 |     "index",
45 |     "x",
46 |     "y",
47 |     "z",

```

在此修改源文件参数

4. 默认模拟时序

上位机命令	模拟回复与行为
01 起飞	立即回复 {clientNo, commandNo}；5 秒后进入 1.5 m 模拟高度，后续周期 0B 反映新高度
02 航线下发	立即回复确认，保存航点，再发送状态 02
03 执行巡检	回复确认；未起飞时等待 5 秒；发送 03；每 5 秒完成一航点并发送 09；发送 07 并等待 5 秒降落；发送 08 和 01
05 一键返航	回复确认并中断旧动作；发送 05；5 秒后到原点；发送 07；5 秒后发送 01
06 降落	回复确认并中断旧动作；发送 07；5 秒后发送 01
07 紧急停机	回复确认并中断旧动作；发送 07；5 秒后发送 01

03 必须先收到一条有效 02 航线；如果没有航线，程序仍按协议回复命令接收确认. 命令 05/06/07 会中断正在进行的巡检，且返航不会发送巡检专用的 08/09。

默认媒体占位值：

- 08.data.videoPath： /home/share/test.mp4
- 08.data.JPGPath： /home/share/jpg
- 09.data.pointPic： /home/share/jpg/test.jpg

注意，本模拟程序不提供或创建真实的视频/图片文件或目录。若要调试FTP功能，需自行创建上述文件，并配置FTP服务和地址以模拟。

电量低于 20% 时发送一次 0C；若模拟飞机已经在空中，还会中断当前任务并模拟返航。

5. 使用上位机或 Postman 调试

上位机测试端连接同一个 WebSocket URL。收到服务端 SYSTEM 后，先订阅模拟客户端的状态主题：

```
1 {"type": "SUBSCRIBE", "topic": "drone/UAV01001/status"}
```

服务端返回 SUB_ACK 后，下发两点巡检航线：

```
1 {"type": "PUBLISH", "topic": "drone/UAV01001/command", "data":
  {"clientNo": "UAV01001", "commandNo": "02", "taskPoints":
    [{"index": 1, "x": 1.0, "y": 0.0, "z": 1.5, "forwardAngle": 0, "cameraAngle": 1495, "photoNo": 1},
    {"index": 2, "x": 2.0, "y": 0.0, "z": 1.5, "forwardAngle": 90, "cameraAngle": 1495, "photoNo": 2}]}}
```

再下发执行命令：

```
1 {"type":"PUBLISH","topic":"drone/UAV01001/command","data":  
  {"clientNo":"UAV01001","commandNo":"03"}}
```

状态订阅端会收到 `BROADCAST` 信封。正常巡检的关键业务顺序是：

```
1 02确认 → 02 → 03确认 → 03 → 09(点1) → 09(点2) → 07 → 08 → 01
```

`0A/0B` 会在上述过程间每秒持续出现。若要查看每个实际收发帧，追加：

```
1 --log-level DEBUG
```

时长、初始位置、电量和媒体路径都可修改，例如快速联调：

```
1 ./venv/bin/python ./gcs-websocket-client.py \  
2 --url ws://127.0.0.1:8581/ws \  
3 --takeoff-seconds 0.2 \  
4 --waypoint-seconds 0.2 \  
5 --landing-seconds 0.2 \  
6 --return-seconds 0.2 \  
7 --power 55.6
```

查看全部参数：

```
1 ./venv/bin/python ./gcs-websocket-client.py --help
```